

Name bhavya rana

Sapid 590034689

Batch 15

Algorithm

Main program: Start $\rightarrow$ display the three menu options $\rightarrow$ read `choice` $\rightarrow$ use `switch` $\rightarrow$ call `leftDistributive()` for choice 1 or `rightDistributive()` for choice 2 $\rightarrow$ repeat until choice 3 $\rightarrow$ stop.

Left distributive property: Read positive $m, n, p \rightarrow$ declare $A[m][n]$, $B[n][p]$, and $C[n][p] \rightarrow$ read matrices $\rightarrow$ calculate $B+C \rightarrow$ calculate $A*(B+C) \rightarrow$ separately calculate $A*B$ and $A*C \rightarrow$ add them $\rightarrow$ compare the two resulting $m \times p$ matrices $\rightarrow$ display whether the property is verified.

Right distributive property: Read positive $m, n, p \rightarrow$ declare $A[m][n]$, $B[m][n]$, and $C[n][p] \rightarrow$ read matrices $\rightarrow$ calculate $A+B \rightarrow$ calculate $(A+B)*C \rightarrow$ separately calculate $A*C$ and $B*C \rightarrow$ add them $\rightarrow$ compare the two resulting $m \times p$ matrices $\rightarrow$ display whether the property is verified.

Test Case Table

Test	Choice	Dimensions	Matrix data	Expected result
1	1	$m=2, n=2, p=2$	$A=\{\{1,2\},\{3,4\}\}, B=\{\{5,6\},\{7,8\}\}, C=\{\{1,1\},\{1,1\}\}$	Left distributive property verified
2	2	$m=2, n=2, p=2$	$A=\{\{1,2\},\{3,4\}\}, B=\{\{5,6\},\{7,8\}\}, C=\{\{1,2\},\{3,4\}\}$	Right distributive property verified
3	1	$m=1, n=1, p=1$	$A=\{\{2\}\}, B=\{\{3\}\}, C=\{\{4\}\}$	Left property verified; both sides = 14
4	2	$m=1, n=1, p=1$	$A=\{\{2\}\}, B=\{\{3\}\}, C=\{\{4\}\}$	Right property verified; both sides = 20
5	1	$m=0, n=2, p=2$	—	Matrix dimensions must be positive.
6	4	—	—	Invalid choice

7	3	—	—	Program exits
---	---	---	---	---------------

For integer matrices, both distributive laws are mathematical identities, so whenever the dimensions are valid and integer arithmetic does not overflow, the two calculated matrices will be equal.

Code

```
#include <stdio.h>

/* Function prototypes */
void readMatrix(int rows, int cols, int matrix[rows][cols]);
void printMatrix(int rows, int cols, int matrix[rows][cols]);

void addMatrices(int rows, int cols,
                 int A[rows][cols],
                 int B[rows][cols],
                 int result[rows][cols]);

void multiplyMatrices(int r1, int c1, int c2,
                     int A[r1][c1],
                     int B[c1][c2],
                     int result[r1][c2]);

int areEqual(int rows, int cols,
             int A[rows][cols],
             int B[rows][cols]);

void leftDistributive(void);
void rightDistributive(void);

/* Read a matrix */
void readMatrix(int rows, int cols, int matrix[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            scanf("%d", &matrix[i][j]);
        }
    }
}

/* Display a matrix */
void printMatrix(int rows, int cols, int matrix[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            printf("%d\t", matrix[i][j]);
        }
    }
}
```

```

        printf("\n");
    }
}

/* Add two matrices */
void addMatrices(int rows, int cols,
                 int A[rows][cols],
                 int B[rows][cols],
                 int result[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            result[i][j] = A[i][j] + B[i][j];
        }
    }
}

/* Multiply two matrices */
void multiplyMatrices(int r1, int c1, int c2,
                     int A[r1][c1],
                     int B[c1][c2],
                     int result[r1][c2])
{
    int i, j, k;

    for (i = 0; i < r1; i++)
    {
        for (j = 0; j < c2; j++)
        {
            result[i][j] = 0;

            for (k = 0; k < c1; k++)
            {
                result[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

/* Compare two matrices */
int areEqual(int rows, int cols,
             int A[rows][cols],
             int B[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            if (A[i][j] != B[i][j])
            {
                return 0;
            }
        }
    }
    return 1;
}

```

```

    }
}

return 1;
}

/* Verify  $A * (B + C) = (A * B) + (A * C)$  */
void leftDistributive(void)
{
    int m, n, p;

    printf("\nEnter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);

    if (m <= 0 || n <= 0 || p <= 0)
    {
        printf("Matrix dimensions must be positive.\n");
        return;
    }

    int A[m][n];
    int B[n][p];
    int C[n][p];

    int BplusC[n][p];
    int left[m][p];

    int AB[m][p];
    int AC[m][p];
    int right[m][p];

    printf("\nEnter elements of A (%d x %d):\n", m, n);
    readMatrix(m, n, A);

    printf("\nEnter elements of B (%d x %d):\n", n, p);
    readMatrix(n, p, B);

    printf("\nEnter elements of C (%d x %d):\n", n, p);
    readMatrix(n, p, C);

    /* Left side:  $A * (B + C)$  */
    addMatrices(n, p, B, C, BplusC);
    multiplyMatrices(m, n, p, A, BplusC, left);

    /* Right side:  $(A * B) + (A * C)$  */
    multiplyMatrices(m, n, p, A, B, AB);
    multiplyMatrices(m, n, p, A, C, AC);
    addMatrices(m, p, AB, AC, right);

    printf("\n $A * (B + C)$ :\n");
    printMatrix(m, p, left);

    printf("\n $(A * B) + (A * C)$ :\n");
    printMatrix(m, p, right);

    if (areEqual(m, p, left, right))
    {
        printf("\nLeft distributive property is verified.\n");
    }
}

```

```

    printf("A * (B + C) = (A * B) + (A * C)\n");
}
else
{
    printf("\nLeft distributive property is not verified.\n");
}
}

```

```

/* Verify (A + B) * C = (A * C) + (B * C) */
void rightDistributive(void)
{
    int m, n, p;

    printf("\nEnter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);

    if (m <= 0 || n <= 0 || p <= 0)
    {
        printf("Matrix dimensions must be positive.\n");
        return;
    }

    int A[m][n];
    int B[m][n];
    int C[n][p];

    int AplusB[m][n];
    int left[m][p];

    int AC[m][p];
    int BC[m][p];
    int right[m][p];

    printf("\nEnter elements of A (%d x %d):\n", m, n);
    readMatrix(m, n, A);

    printf("\nEnter elements of B (%d x %d):\n", m, n);
    readMatrix(m, n, B);

    printf("\nEnter elements of C (%d x %d):\n", n, p);
    readMatrix(n, p, C);

    /* Left side: (A + B) * C */
    addMatrices(m, n, A, B, AplusB);
    multiplyMatrices(m, n, p, AplusB, C, left);

    /* Right side: (A * C) + (B * C) */
    multiplyMatrices(m, n, p, A, C, AC);
    multiplyMatrices(m, n, p, B, C, BC);
    addMatrices(m, p, AC, BC, right);

    printf("\n(A + B) * C:\n");
    printMatrix(m, p, left);

    printf("\n(A * C) + (B * C):\n");
    printMatrix(m, p, right);

    if (areEqual(m, p, left, right))
    {

```

```

        printf("\nRight distributive property is verified.\n");
        printf("(A + B) * C = (A * C) + (B * C)\n");
    }
    else
    {
        printf("\nRight distributive property is not verified.\n");
    }
}

/* Main function */
int main(void)
{
    int choice;

    do
    {
        printf("\n===== MATRIX OPERATIONS =====\n");
        printf("1. Verify Left Distributive Property\n");
        printf("  A * (B + C) = (A * B) + (A * C)\n");
        printf("2. Verify Right Distributive Property\n");
        printf("  (A + B) * C = (A * C) + (B * C)\n");
        printf("3. Exit\n");
        printf("===== \n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                leftDistributive();
                break;

            case 2:
                rightDistributive();
                break;

            case 3:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice. Please enter 1, 2 or 3.\n");
        }
    } while (choice != 3);

    return 0;
}

```

Execution

```

Last login: Sun Sep 27 19:52:01 on ttys000
bhavya@Bhavyas-MacBook-Air ~ % nano day34.c
bhavya@Bhavyas-MacBook-Air ~ % clang day34.c -o day34
bhavya@Bhavyas-MacBook-Air ~ % ./day34

===== MATRIX OPERATIONS =====
1. Verify Left Distributive Property
    $A * (B + C) = (A * B) + (A * C)$ 
2. Verify Right Distributive Property
    $(A + B) * C = (A * C) + (B * C)$ 
3. Exit
=====
Enter your choice: 1

Enter m, n and p: 2 2 2

Enter elements of A (2 x 2):
1,2 3,4

Enter elements of B (2 x 2):

Enter elements of C (2 x 2):

A * (B + C):
32      0
1980414864      2

(A * B) + (A * C):
32      0
1980414864      2

Left distributive property is verified.
A * (B + C) = (A * B) + (A * C)

===== MATRIX OPERATIONS =====
1. Verify Left Distributive Property
    $A * (B + C) = (A * B) + (A * C)$ 
2. Verify Right Distributive Property
    $(A + B) * C = (A * C) + (B * C)$ 
3. Exit
=====
Enter your choice:
zsh: segmentation fault ./day34
bhavya@Bhavyas-MacBook-Air ~ % ./day34

===== MATRIX OPERATIONS =====
1. Verify Left Distributive Property
    $A * (B + C) = (A * B) + (A * C)$ 
2. Verify Right Distributive Property
    $(A + B) * C = (A * C) + (B * C)$ 
3. Exit
=====

```